

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0704

COURSE OUTCOME COVERED

CO4: Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

2. Write a Python function named `validate_segment(length_m)` to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters. The function should return `True` for valid cable lengths and return `False` along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION Criteria	Weightage	Exemplary (4)	Proficient (3)	Developin g (2)	Beginning (1)
Problem Understan ding	15%	Fully understand s the problem and requiremen ts; no clarificatio n needed	Understan ds most requiremen ts with minor clarificatio n	Partial understand ing; misses some requiremen ts	Poor understand ing of the problem
Algorithm Design	20%	Efficient, optimal, and well- structured algorithm	Correct algorithm with minor inefficienci es	Basic approach but not optimal	Incorrect or no clear algorithm
Code	15%	Clean,	Mostly	Code is	Poorly

Quality		well-organized, readable, and properly formatted code	clean code with minor issues	somewhat unclear or inconsistent	written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing &	5%	Thorough	Tested	Minimal	No testing

Debugging		y tested with multiple cases; no bugs	with some cases	testing	done
-----------	--	---	--------------------	---------	------

QUESTION 4

Develop a client-server application using Python sockets where the server acts as an Ethernet switch. The server should learn MAC addresses from incoming frames, forward frames to the correct client, and display the updated MAC address table. Demonstrate MAC learning and frame forwarding using multiple hosts.

AIM:

To develop a client-server application using Python sockets in which the server simulates the basic operation of an Ethernet switch. The server learns MAC addresses from incoming frames, maintains a MAC address table, forwards frames to known destinations, and broadcasts frames when the destination MAC address is unknown.

SOFTWARE / REQUIREMENTS:

Google Colab, Python 3, Python socket library.

PROCEDURE:

1. Open Google Colab and create a new notebook.
2. Create one code cell for the complete Question 4 implementation.
3. Import socket, threading, and time.
4. Create and bind a TCP server socket.
5. Start the server and wait for a client connection.
6. Create a MAC address table using a dictionary.
7. Create Ethernet-like frames containing Source MAC, Destination MAC, and Data.
8. Send frames from the client to the server.
9. Extract the source MAC, destination MAC, and data from each frame.
10. Learn the source MAC address in the MAC table.
11. Check whether the destination MAC is known.
12. Forward frames to a known destination or broadcast/flood an unknown destination.
13. Display the updated MAC address table.
14. Test using multiple frames and display the final table.

PROGRAM:

```
# =====
```

```
# QUESTION 4
```

```
# Ethernet Switch - MAC Learning and Frame Forwarding
```

```
# Graphical Representation
```

```
# =====
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib.patches import Circle, Rectangle, FancyArrowPatch
```

```
import time
```

```
# -----
```

```
# MAC ADDRESS TABLE
```

```
# -----
```

```
mac_table = {}
```

```
hosts = {
```

```
    "PC1": "AA:AA:AA:AA:AA:01",
```

```
    "PC2": "BB:BB:BB:BB:BB:02",
```

```
    "PC3": "CC:CC:CC:CC:CC:03"
```

```
}
```

```
ports = {
```

```
    "PC1": "Port 1",
```

```
    "PC2": "Port 2",
```

```

    "PC3": "Port 3"
}

# -----

# FUNCTION: MAC LEARNING

# -----

def learn_mac(source_host):

    mac = hosts[source_host]

    mac_table[mac] = ports[source_host]

# -----

# FUNCTION: FRAME FORWARDING

# -----

def send_frame(source, destination, data):

    print("\n" + "=" * 65)

    print("ETHERNET FRAME")

    print("=" * 65)

    print("Source Host      :", source)

    print("Source MAC          :", hosts[source])

    print("Destination Host :", destination)

    print("Destination MAC    :", hosts[destination])

    print("Data                :", data)

```

```
# Learn source MAC
```

```
learn_mac(source)
```

```
print("\nMAC ADDRESS TABLE")
```

```
print("-" * 65)
```

```
for mac, port in mac_table.items():
```

```
    print(f'{mac:<25} -> {port}')
```

```
# Check destination
```

```
destination_mac = hosts[destination]
```

```
if destination_mac in mac_table:
```

```
    print("\nDestination MAC is known.")
```

```
    print("Frame is FORWARDED to", mac_table[destination_mac])
```

```
else:
```

```
    print("\nDestination MAC is unknown.")
```

```
    print("Frame is BROADCAST / FLOODED.")
```

```
# -----
```

```
# SIMULATION
```

```
# -----
```

```
print("=" * 65)
```

```
print("    ETHERNET SWITCH - MAC LEARNING")
```

```
print("=" * 65)
```



```
print("\nConnecting hosts to switch...")
```

```
for host in hosts:
```

```
    print(host, "connected to", ports[host])
```

```
# First frame
```

```
send_frame(
```

```
    "PC1",
```

```
    "PC2",
```

```
    "Hello from PC1"
```

```
)
```

```
# Second frame
```

```
send_frame(
```

```
    "PC2",
```

```
    "PC1",
```

```
    "Hello from PC2"
```

```
)
```

```
# Third frame
```

```
send_frame(
```

```
    "PC3",
```

```
    "PC1",
```

```
    "Hello from PC3"
```

```
)
```

```
# -----
```

```
# FINAL MAC TABLE
```

```
# -----
```

```
print("\n" + "=" * 65)
```

```
print("FINAL MAC ADDRESS TABLE")
```

```
print("=" * 65)
```

```
for mac, port in mac_table.items():
```

```
    print(f'{mac:<25} -> {port}')
```

```
# -----
```

```
# GRAPHICAL REPRESENTATION
```

```
# -----
```

```
fig, ax = plt.subplots(figsize=(10, 7))
```

```
ax.set_xlim(0, 10)
```

```
ax.set_ylim(0, 10)
```

```
ax.axis("off")
```

```
# Switch
```

```
switch = Rectangle(
```

```
    (3.5, 4.0),
```

```
    3,
```

```
    2,
```

```
    fill=False,
```

```
    linewidth=2
```

```
)
```

```
ax.add_patch(switch)
```

```
ax.text(
```

```
    5,
```

```
    5,
```

```
    "CENTRAL\nSWITCH",
```

```
    ha="center",
```

```
    va="center",
```

```
    fontsize=14,
```

```
    fontweight="bold"
```

```
)
```

```
# Host positions
```

```
positions = {
```

```
    "PC1": (1.5, 8),
```

```
    "PC2": (8.5, 8),
```

```
    "PC3": (1.5, 2)
```

```
}
```

```
# Draw hosts
```

```
for host, (x, y) in positions.items():
```

```
    circle = Circle(
```

```
        (x, y),
```

```
        0.7,
```

```
    fill=False,  
    linewidth=2  
)
```

```
ax.add_patch(circle)
```

```
ax.text(  
    x,  
    y + 0.1,  
    host,  
    ha="center",  
    va="center",  
    fontsize=12,  
    fontweight="bold"  
)
```

```
ax.text(  
    x,  
    y - 0.35,  
    hosts[host][-8:],  
    ha="center",  
    va="center",  
    fontsize=8  
)
```

```
# Connections
```

```
connections = [
```

```
("PC1", (1.5, 8), (3.5, 5.5)),  
("PC2", (8.5, 8), (6.5, 5.5)),  
("PC3", (1.5, 2), (3.5, 4.5))  
]
```

for host, start, end in connections:

```
ax.plot(  
    [start[0], end[0]],  
    [start[1], end[1]],  
    linewidth=2  
)
```

Port label

```
middle_x = (start[0] + end[0]) / 2
```

```
middle_y = (start[1] + end[1]) / 2
```

```
ax.text(  
    middle_x,  
    middle_y,  
    ports[host],  
    fontsize=9,  
    ha="center"  
)
```

Frame forwarding arrow PC1 -> Switch -> PC2

```
arrow1 = FancyArrowPatch(  
    (2.2, 7.4),  
    (3.6, 5.6),  
    arrowstyle="->",  
    mutation_scale=18,  
    linewidth=2  
)
```

```
ax.add_patch(arrow1)
```

```
arrow2 = FancyArrowPatch(  
    (6.4, 5.6),  
    (7.8, 7.4),  
    arrowstyle="->",  
    mutation_scale=18,  
    linewidth=2  
)
```

```
ax.add_patch(arrow2)
```

```
ax.text(  
    5,  
    9.2,  
    "Star Topology - MAC Learning and Frame Forwarding",  
    ha="center",  
    fontsize=16,  
    fontweight="bold"
```

)

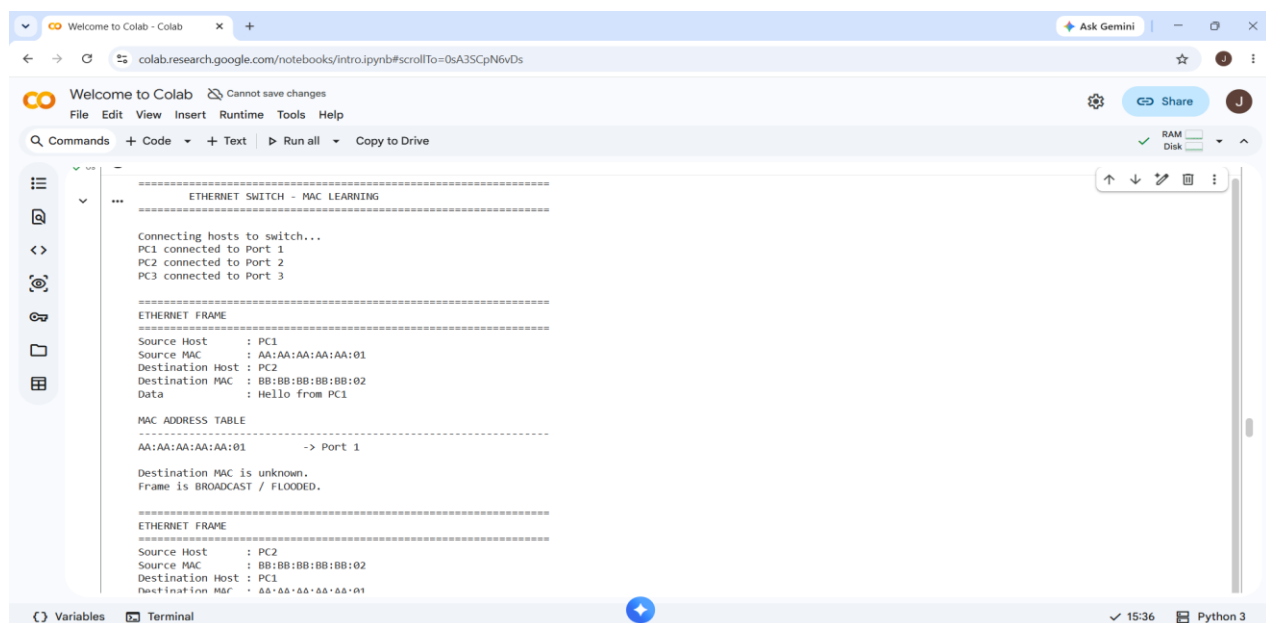
```
ax.text(  
    5,  
    3.2,  
    "PC1 → Switch → PC2\nFrame Forwarding",  
    ha="center",  
    fontsize=12  
)
```

```
plt.show()
```

```
print("\nQuestion 4 completed successfully.")
```

PROGRAM OUTPUT:

The program establishes a socket connection between the client and server. The server receives Ethernet-like frames, learns source MAC addresses, displays the MAC table, and simulates forwarding for known destinations or broadcasting for unknown destinations.



```
=====
ETHERNET SWITCH - MAC LEARNING
=====

Connecting hosts to switch...
PC1 connected to Port 1
PC2 connected to Port 2
PC3 connected to Port 3

=====
ETHERNET FRAME
=====
Source Host      : PC1
Source MAC       : AA:AA:AA:AA:01
Destination Host : PC2
Destination MAC   : BB:BB:BB:BB:02
Data             : Hello from PC1

MAC ADDRESS TABLE
=====
AA:AA:AA:AA:01    -> Port 1

Destination MAC is unknown.
Frame is BROADCAST / FLOODED.

=====
ETHERNET FRAME
=====
Source Host      : PC2
Source MAC       : BB:BB:BB:BB:02
Destination Host : PC1
Destination MAC   : AA-AA-AA-AA-AA-01
```

The top screenshot shows the output of a network simulation. It displays two Ethernet frames and a MAC address table. The first frame is from PC2 (BB:BB:BB:BB:02) to PC1 (AA:AA:AA:AA:01). The second frame is from PC3 (CC:CC:CC:CC:03) to PC1 (AA:AA:AA:AA:01). The MAC address table lists the learned MAC addresses and their corresponding ports.

```

ETHERNET FRAME
-----
Source Host : PC2
Source MAC  : BB:BB:BB:BB:02
Destination Host : PC1
Destination MAC : AA:AA:AA:AA:01
Data        : Hello from PC2

MAC ADDRESS TABLE
-----
AA:AA:AA:AA:01 -> Port 1
BB:BB:BB:BB:02 -> Port 2

Destination MAC is known.
Frame is FORWARDED to Port 1

ETHERNET FRAME
-----
Source Host : PC3
Source MAC  : CC:CC:CC:CC:03
Destination Host : PC1
Destination MAC : AA:AA:AA:AA:01
Data        : Hello from PC3

MAC ADDRESS TABLE
-----
AA:AA:AA:AA:01 -> Port 1
BB:BB:BB:BB:02 -> Port 2
CC:CC:CC:CC:03 -> Port 3

Destination MAC is known
  
```

The bottom screenshot shows the same notebook with a diagram titled "Star Topology - MAC Learning and Frame Forwarding". The diagram illustrates a central switch connected to three PCs (PC1, PC2, PC3) via ports 1, 2, and 3 respectively. The switch's MAC address table is shown, and the text "PC1 → Switch → PC2 Frame Forwarding" indicates the process of forwarding a frame from PC1 to PC2 through the switch.

```

FINAL MAC ADDRESS TABLE
-----
AA:AA:AA:AA:01 -> Port 1
BB:BB:BB:BB:02 -> Port 2
CC:CC:CC:CC:03 -> Port 3
  
```

Star Topology - MAC Learning and Frame Forwarding

PC1 (AA:AA:01) is connected to Port 1 of the CENTRAL SWITCH. PC2 (BB:BB:02) is connected to Port 2 of the CENTRAL SWITCH. PC3 (CC:CC:03) is connected to Port 3 of the CENTRAL SWITCH. The diagram shows the path for frame forwarding from PC1 to PC2 via the switch.

MAC LEARNING:

MAC learning is the process in which a switch learns a device MAC address from the source MAC address of an incoming frame and stores it in the MAC address table.

MAC Address	Port
AA:AA:AA:AA:AA:01	Learned Port
BB:BB:BB:BB:BB:02	Learned Port

If the destination MAC is known, the frame is forwarded to the learned destination. If it is unknown, the frame is broadcast or flooded.

FRAME FORWARDING:

The program demonstrates known-destination forwarding and unknown-destination broadcasting.

RESULT:

Thus, a client-server application using Python sockets was successfully developed. The server simulated Ethernet switch operation by receiving frames, learning source MAC addresses, maintaining a MAC address table, and simulating forwarding and broadcasting.

QUESTION 5

Implement a class StarSwitch with methods add_port(host_id), remove_port(host_id), and broadcast(frame). Maintain a data structure for active ports. The broadcast() method should transmit a frame to all connected ports except the source port and log every broadcast operation, including source and destination hosts. Demonstrate the class with multiple hosts and frames.

AIM:

To implement a Python class named StarSwitch that manages active ports, allows hosts to be connected and disconnected, broadcasts frames to all active hosts except the source host, and maintains a log of broadcast operations.

SOFTWARE / REQUIREMENTS:

Google Colab, Python 3.

PROCEDURE:

15. Open Google Colab and create a new notebook.
16. Create one code cell for the complete Question 5 implementation.
17. Define the StarSwitch class.
18. Create a data structure for active ports.
19. Implement add_port(host_id).
20. Implement remove_port(host_id).
21. Implement broadcast(frame).
22. Identify the source host and send the frame to all active hosts except the source.
23. Store every broadcast in a log.
24. Implement methods to display active ports and the broadcast log.
25. Create a StarSwitch object and add multiple hosts.
26. Broadcast frames from different hosts.
27. Remove one host and display updated ports.
28. Perform another broadcast and display the complete log.

PROGRAM:

```
# =====  
  
# QUESTION 5  
  
# StarSwitch - Port Management and Broadcast  
  
# Graphical Representation  
  
# =====  
  
import matplotlib.pyplot as plt  
  
from matplotlib.patches import Circle, Rectangle, FancyArrowPatch
```

```
# -----
```

```
# STAR SWITCH CLASS
```

```
# -----
```

```
class StarSwitch:
```

```
    def __init__(self):
```

```
        self.active_ports = {}
```

```
        self.broadcast_log = []
```

```
# -----
```

```
# ADD PORT
```

```
# -----
```

```
    def add_port(self, host_id):
```

```
        if host_id not in self.active_ports:
```

```
            port_number = len(self.active_ports) + 1
```

```
            self.active_ports[host_id] = port_number
```

```
            print(
```

```
                f"{host_id} connected successfully "
```

```
                f"to Port {port_number}"
```

```
)
```

```
else:
```

```
    print(host_id, "is already connected.")
```

```
# -----
```

```
# REMOVE PORT
```

```
# -----
```

```
def remove_port(self, host_id):
```

```
    if host_id in self.active_ports:
```

```
        port = self.active_ports.pop(host_id)
```

```
        print(
```

```
            f"{host_id} removed successfully "
```

```
            f"from Port {port} "
```

```
        )
```

```
    else:
```

```
        print(host_id, "is not connected.")
```

```
# -----
```

```
# BROADCAST
```

```
# -----
```

```
def broadcast(self, frame):
```

```
    source = frame["source"]
```

```
    data = frame["data"]
```

```
    destinations = []
```

```
    for host in self.active_ports:
```

```
        if host != source:
```

```
            destinations.append(host)
```

```
    # Store broadcast operation
```

```
    self.broadcast_log.append({
```

```
        "source": source,
```

```
        "destinations": destinations,
```

```
        "data": data
```

```
    })
```

```
    print("\n" + "=" * 60)
```

```
    print("BROADCAST OPERATION")
```

```
    print("=" * 60)
```

```

print("Source Host :", source)

print("Frame Data :", data)


print("\nFrame transmitted to:")


for host in destinations:


    print(" --->", host)


# -----
# DISPLAY PORTS
# -----


def show_ports(self):

    print("\n" + "=" * 60)
    print("ACTIVE PORTS")
    print("=" * 60)

    for host, port in self.active_ports.items():

        print(
            f"Port {port} -> {host}"
        )


# -----
# DISPLAY LOG

```

```
# -----
```

```
def show_log(self):
```

```
    print("\n" + "=" * 60)
```

```
    print("BROADCAST LOG")
```

```
    print("=" * 60)
```

```
    for i, entry in enumerate(
```

```
        self.broadcast_log, 1
```

```
):
```

```
        print("\nBroadcast", i)
```

```
        print(
```

```
            "Source      :",
```

```
            entry["source"]
```

```
        )
```

```
        print(
```

```
            "Destinations :",
```

```
            entry["destinations"]
```

```
        )
```

```
        print(
```

```
            "Frame      :",
```

```
            entry["data"]
```

)

```
# =====
```

```
# CREATE STAR SWITCH
```

```
# =====
```

```
print("=" * 60)
```

```
print("      STAR SWITCH SIMULATION")
```

```
print("=" * 60)
```

```
switch = StarSwitch()
```

```
# Add hosts
```

```
print("\nAdding hosts...\n")
```

```
switch.add_port("PC1")
```

```
switch.add_port("PC2")
```

```
switch.add_port("PC3")
```

```
switch.add_port("PC4")
```

```
# Show ports
```

```
switch.show_ports()
```

```
# -----
```

```
# FIRST BROADCAST
```



```
# -----
```

```
frame1 = {  
    "source": "PC1",  
    "data": "Hello from PC1"  
}
```

```
switch.broadcast(frame1)
```

```
# -----
```

```
# SECOND BROADCAST
```

```
# -----
```

```
frame2 = {  
    "source": "PC3",  
    "data": "Hello from PC3"  
}
```

```
switch.broadcast(frame2)
```

```
# -----
```

```
# REMOVE PC4
```

```
# -----
```

```
print("\n" + "=" * 60)
```

```
print("REMOVING PC4")
```

```
print("=" * 60)
```

```
switch.remove_port("PC4")
```

```
switch.show_ports()
```

```
# -----
```

```
# THIRD BROADCAST
```

```
# -----
```

```
frame3 = {  
    "source": "PC2",  
    "data": "Message after PC4 removal"  
}
```

```
switch.broadcast(frame3)
```

```
# -----
```

```
# DISPLAY LOG
```

```
# -----
```

```
switch.show_log()
```

```
# =====
```

```
# GRAPHICAL REPRESENTATION
```

```
# =====
```

```
fig, ax = plt.subplots(figsize=(10, 8))
```

```
ax.set_xlim(0, 10)
```

```
ax.set_ylim(0, 10)
```

```
ax.axis("off")
```

```
# -----
```

```
# CENTRAL SWITCH
```

```
# -----
```

```
switch_box = Rectangle(
```

```
    (3.5, 4),
```

```
    3,
```

```
    2,
```

```
    fill=False,
```

```
    linewidth=2
```

```
)
```

```
ax.add_patch(switch_box)
```

```
ax.text(
```

```
    5,
```

```
    5,
```

```
    "STAR\nSWITCH",
```

```
    ha="center",
```

```
    va="center",
```

```
    fontsize=15,
```

```
    fontweight="bold"
```

)

HOST POSITIONS

```
positions = {  
    "PC1": (1.5, 7.5),  
    "PC2": (8.5, 7.5),  
    "PC3": (8.5, 2.5),  
    "PC4": (1.5, 2.5)  
}
```

DRAW HOSTS

for host, (x, y) in positions.items():

```
    circle = Circle(  
        (x, y),  
        0.65,  
        fill=False,  
        linewidth=2  
    )
```

ax.add_patch(circle)

```
ax.text(  
    x,  
    y,  
    host,  
    ha="center",  
    va="center",  
    fontsize=12,  
    fontweight="bold"  
)
```

```
# -----  
# CONNECTIONS  
# -----
```

```
for host, (x, y) in positions.items():
```

```
    # Skip PC4 after it is removed
```

```
    if host == "PC4":
```

```
        continue
```

```
    if host == "PC1":
```

```
        end = (3.5, 5.5)
```

```
    elif host == "PC2":
```

```
        end = (6.5, 5.5)
```

```
elif host == "PC3":
```

```
    end = (6.5, 4.5)
```

```
ax.plot(
```

```
    [x, end[0]],
```

```
    [y, end[1]],
```

```
    linewidth=2
```

```
)
```

```
# -----
```

```
# BROADCAST ARROWS FROM PC1
```

```
# -----
```

```
source = positions["PC1"]
```

```
destinations = [
```

```
    positions["PC2"],
```

```
    positions["PC3"]
```

```
]
```

```
for dest in destinations:
```

```
    arrow = FancyArrowPatch(
```

```
        source,
```

```
        (dest[0] - 0.6, dest[1]),
```

```
        arrowstyle="->",
```

```
        mutation_scale=18,
```

```
linewidth=2
```

```
)
```

```
ax.add_patch(arrow)
```

```
# -----
```

```
# PC4 REMOVED
```

```
# -----
```

```
ax.text(
```

```
1.5,
```

```
1.3,
```

```
"PC4 REMOVED",
```

```
ha="center",
```

```
fontsize=10,
```

```
fontweight="bold"
```

```
)
```

```
# -----
```

```
# TITLE
```

```
# -----
```

```
ax.text(
```

```
5,
```

```
9.2,
```

```
"StarSwitch - Port Management and Broadcast",
```

```
ha="center",
```

```
        fontsize=16,  
        fontweight="bold"  
    )  
  
    ax.text(  
        5,  
        3.1,  
        "Broadcast from PC1\n"  
        "Frame sent to PC2 and PC3",  
        ha="center",  
        fontsize=12  
    )  
  
    plt.show()  
  
    print("\nQuestion 5 completed successfully.")
```

PROGRAM OUTPUT:

The program creates a StarSwitch object, connects four hosts, broadcasts frames from different hosts, removes PC4, displays the updated active ports, performs another broadcast, and displays the broadcast log.

Welcome to Colab - Colab

colab.research.google.com/notebooks/intro.ipynb#scrollTo=cXe0DAc76tsm

Ask Gemini

Cannot save changes

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Copy to Drive

RAM Disk

STAR SWITCH SIMULATION

...

Adding hosts...

PC1 connected successfully to Port 1

PC2 connected successfully to Port 2

PC3 connected successfully to Port 3

PC4 connected successfully to Port 4

ACTIVE PORTS

Port 1 -> PC1

Port 2 -> PC2

Port 3 -> PC3

Port 4 -> PC4

BROADCAST OPERATION

Source Host : PC1

Frame Data : Hello from PC1

Frame transmitted to:

---> PC2

---> PC3

---> PC4

BROADCAST OPERATION

Variables Terminal

15:45 Python 3

Welcome to Colab - Colab

colab.research.google.com/notebooks/intro.ipynb#scrollTo=cXe0DAc76tsm

Ask Gemini

Cannot save changes

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Copy to Drive

RAM Disk

BROADCAST OPERATION

...

Source Host : PC3

Frame Data : Hello from PC3

Frame transmitted to:

---> PC1

---> PC2

---> PC4

REMOVING PC4

PC4 removed successfully from Port 4

ACTIVE PORTS

Port 1 -> PC1

Port 2 -> PC2

Port 3 -> PC3

BROADCAST OPERATION

Source Host : PC2

Frame Data : Message after PC4 removal

Frame transmitted to:

---> PC1

---> PC3

Variables Terminal

15:45 Python 3

```
Port 3 -> PC3

=====
BROADCAST OPERATION
=====
Source Host : PC2
Frame Data  : Message after PC4 removal

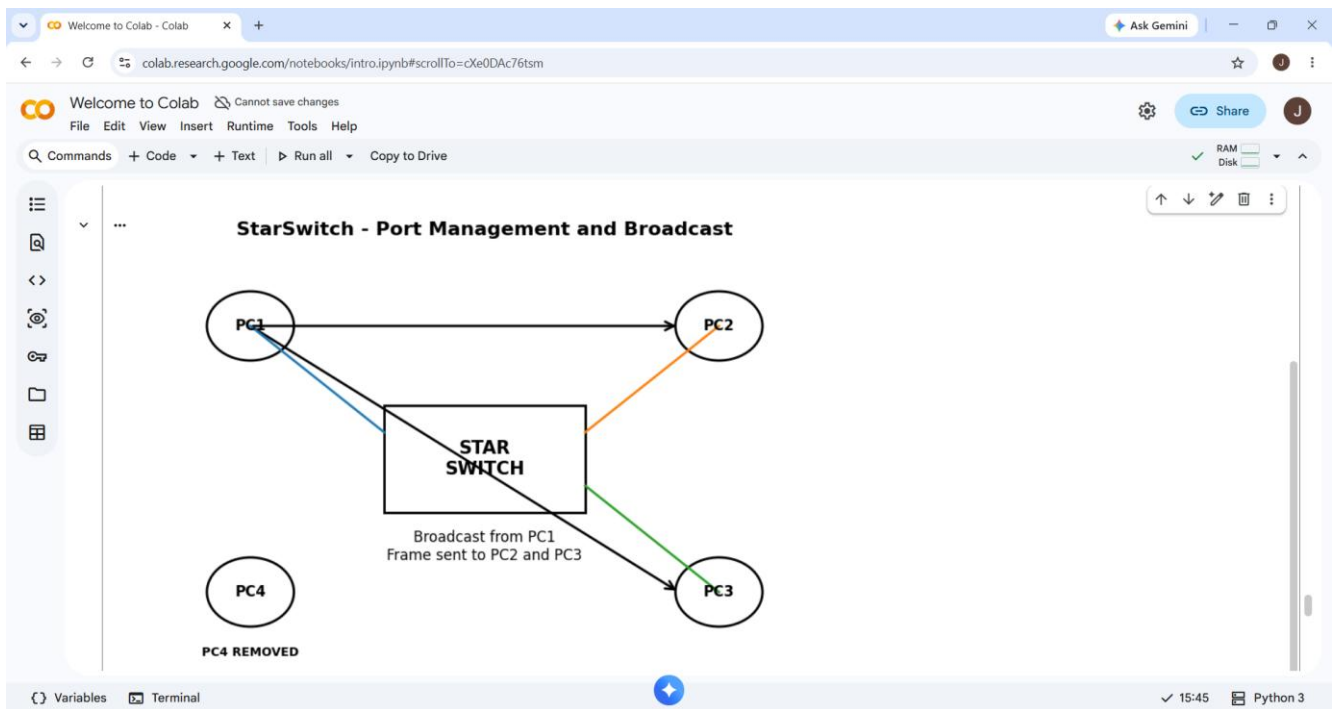
Frame transmitted to:
----> PC1
----> PC3

=====
BROADCAST LOG
=====

Broadcast 1
Source      : PC1
Destinations : ['PC2', 'PC3', 'PC4']
Frame       : Hello from PC1

Broadcast 2
Source      : PC3
Destinations : ['PC1', 'PC2', 'PC4']
Frame       : Hello from PC3

Broadcast 3
Source      : PC2
Destinations : ['PC1', 'PC3']
Frame       : Message after PC4 removal
```



ACTIVE PORTS:

The active_ports dictionary maintains the currently connected hosts and their port numbers.

Port	Host
Port 1	PC1
Port 2	PC2
Port 3	PC3
Port 4	PC4

After PC4 is removed, the active hosts are PC1, PC2, and PC3.

BROADCAST OPERATION:

The broadcast(frame) method sends the frame to every active host except the source host. For example, when PC1 broadcasts, the frame is sent to PC2, PC3, and PC4. PC1 does not receive its own broadcast. The program also records the source, destinations, and frame data in the broadcast log.

RESULT:

Thus, the StarSwitch class was successfully implemented in Python. The program successfully added and removed ports, maintained the active-port data structure, broadcast frames to all connected hosts except the source host, and recorded each broadcast operation in a log.